ASP.NET Core Web API + EF Core Master Concept Guide

Introduction

This guide explains the concepts behind your Field Service Management project in simple language. Lea

Project Architecture

```
React Frontend
    ↓ HTTP
ASP.NET Core Web API
    ↓
Controller
    ↓
Service (later)
    ↓
Entity Framework Core
    ↓
DbContext
    ↓
SQL Server
```

Dependency Injection (DI)

DI means objects are created and provided by ASP.NET Core instead of you creating them manually. Exa

Program.cs

Program.cs is the entry point of the application. It builds the app, reads appsettings.json, registe

Builder and Services

builder creates the application configuration. builder.Services is the Dependency Injection containe

appsettings.json

Stores configuration like connection strings and logging. The application reads these settings throu

AppDbContext

AppDbContext inherits from DbContext. It represents a session with the database. DbSet<User> tells E

Model / Entity

A Model (Entity) is a C# class representing a database table. Properties become table columns. Examp

DbSet

DbSet<User> represents the Users table and allows querying and saving User objects.

Entity Framework Core

EF Core is an ORM (Object Relational Mapper). It converts C# objects into SQL queries automatically.

ADO.NET vs EF Core

ADO.NET requires writing SQL commands manually, opening connections, creating SqlCommand and SqlData

Code First Workflow

```
Create Model
      ↓
Add DbSet
      ↓
Create Migration
      ↓
Update Database
      ↓
SQL Tables Created
```

Migration

A migration is a snapshot of model changes. 'dotnet ef migrations add InitialCreate' generates SQL cl

Controller

Controllers receive HTTP requests, validate input, call EF Core or Services, and return HTTP respons

HTTP Methods

```
GET = Read data
POST = Create data
PUT = Update data
DELETE = Remove data
```

Request Lifecycle

```
React Form
   ↓
Axios fetch()
   ↓
HTTP Request
   ↓
Controller
   ↓
AppDbContext
   ↓
EF Core
   ↓
SQL Server
   ↓
Response JSON
   ↓
React UI Updates
```

Frontend to Backend

React collects form data, sends JSON through Axios. ASP.NET Core binds JSON to a C# object, saves it

Your Project Workflow

```
Customer Login
↓
Create Service Request
↓
Manager Reviews
↓
Assign Technician
↓
Technician Accepts
↓
Starts Travel
↓
Starts Work
↓
Submits Work Report
↓
Manager Approves
↓
Generate Invoice
↓
```

```
Customer Pays
↓
Close Request
```

Important Commands

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet ef migrations add InitialCreate
dotnet ef database update
dotnet run
```

Interview Questions

```
What is DI? What is DbContext? What is DbSet? What is EF Core? Difference between EF Core and ADO.NE
```